First line: String *A*

Next line: String *B*

Output format

For each test case, print *YES* if string *A* can be converted to string *B*, otherwise print *NO*

Constraints

$1 \leqslant len(A) \leqslant 1000000$

$1 \leqslant len(B) \leqslant 1000000$

SAMPLE INPUT

```
abaca
cdbda
```

SAMPLE OUTPUT

```
YES
```

Explanation

The string ***abaca*** can be converted to ***bcbda*** in one move and to ***cdbda*** in the next move.

Answer: (penalty regime: 0 %)

```c
#include <stdio.h>
#include <string.h>

int are_compatible(const char* A, const c
    int lenA = strlen(A);
    int lenB = strlen(B);

    if (lenA != lenB) {
        return 0;
    }

    for (int i = 0; i < lenA; ++i) {
        if (A[i] > B[i]) {
            return 0;
        }
    }
    return 1;
}

int main() {
    char A[1000001], B[1000001];

    scanf("%s", A);
    scanf("%s", B);

    if (are_compatible(A, B)) {
        printf("YES\n");
    } else {
        printf("NO\n");
    }

    return 0;
}
```

	Input	Expected	Got	
✓	abaca cdbda	YES	YES	✓

Passed all tests! ✓

English alphabet

OUTPUT

The first and only line of output must contain the length of the correct password and its central letter

CONSTRAINTS

$1 \leq N \leq 100$

SAMPLE INPUT

4
abc
def
feg
cba

SAMPLE OUTPUT

3 b

Answer: (penalty regime: 0 %)

```
#include <stdio.h>
#include <string.h>

// Function to reverse a string
void reverse(char str[], char rev[]) {
    int len = strlen(str);
    for (int i = 0; i < len; i++) {
        rev[i] = str[len - i - 1];
    }
    rev[len] = '\0';
}

int main() {
    int n;
    char passwords[100][15]; // Max 100 p

    // Input the number of passwords
    scanf("%d", &n);

    // Input the passwords
    for (int i = 0; i < n; i++) {
        scanf("%s", passwords[i]);
    }

    // Iterate through the passwords to f
    for (int i = 0; i < n; i++) {
        char reversed[15];
        reverse(passwords[i], reversed);

        // Check if the reverse exists in
        for (int j = 0; j < n; j++) {
            if (strcmp(reversed, password
                int len = strlen(password
                printf("%d %c\n", len, pa
                return 0; // Exit after f
            }
        }
    }

    return 0;
}
```

	Input	Expected	Got	
✓	4 abc def feg cba	3 b	3 b	✓

Passed all tests! ✓

First line has N, the total number of restaurants.

Next N lines contain Name of Restaurant and Points awarded by Joey, separated by a space. Restaurant name has no spaces, all lowercase letters and will not be more than 20 characters.

Output

Print the name of the restaurant that Joey should choose

Constraints:

$1 <= N <= 10^5$

$1 <= Points <= 10^6$

SAMPLE INPUT

3

Pizzeria 108

Dominos 145

Pizzapizza 49

SAMPLE OUTPUT

Dominos

Explanation

Dominos has maximum points.

Answer: (penalty regime: 0 %)

```c
#include <stdio.h>
#include <string.h>

int main() {
    int N;
    scanf("%d", &N);

    char best_restaurant[21];  // To stor
    int max_points = -1;

    for (int i = 0; i < N; i++) {
        char restaurant_name[21];
        int points;

        // Input the restaurant name and
        scanf("%s %d", restaurant_name, &

        // Determine if we need to update
        if (points > max_points || (point
            max_points = points;
            strcpy(best_restaurant, resta
        }
    }

    // Output the best restaurant's name
    printf("%s\n", best_restaurant);

    return 0;
}
```

	Input	Expected	Got	
✓	3 Pizzeria 108 Dominos 145 Pizzapizza 49	Dominos	Dominos	✓

Passed all tests! ✓

[illegible]

[illegible]

Output

Print "YES" if it is valid mobile number else print "NO"

Note: Quotes are for clarity

Constraints

1<= T <= $10^{[illegible]}$

sum of string length <= 10^{5}

SAMPLE INPUT

3

1234567890

0123456789

0123456.87

SAMPLE OUTPUT

YES

NO

NO

Answer: (penalty regime: 0 %)

```c
#include <stdio.h>
#include <string.h>
#include <ctype.h>

int is_valid_number(char s[]) {
    int len = strlen(s);
    if (len != 10) {
        return 0;
    }
    if (s[0] == '0') {
        return 0;
    }
    for (int i = 0; i < len; i++) {
        if (!isdigit(s[i])) {
            return 0;
        }
    }
    return 1;
}

int main() {
    int T;
    scanf("%d", &T);

    for (int i = 0; i < T; i++) {
        char S[101];
        scanf("%s", S);
        if (is_valid_number(S)) {
            printf("YES\n");
        } else {
            printf("NO\n");
        }
    }
    return 0;
}

```

	Input	Expected	Got	
✓	3 1234567890 0123456789 0123456.87	YES NO NO	YES NO NO	✓

Passed all tests! ✓